



ÇANKAYA UNIVERSITY
Software Engineering Department



SENG 491 – 492 Graduation Project

Software Requirements

Specification

Dynamic Port Switching

Eren Abdullah BEŞİK – 202128019

Şahin ERŞAN – 202128002

Berfin Duru ALKAN – 202228005

Version <<1.2>>

Table of Contents

1	Introduction.....	1
1.1	Purpose.....	1
1.2	Scope	1
1.3	Threat Model and Defense Objectives	2
1.3.1	In-Scope Threats (Mitigated Attacks)	2
1.3.2	Out-of-Scope Threats (Not Mitigated)	3
1.4	Definitions, acronyms, and abbreviations	3
1.5	Overview	4
1.6	Version History	4
2	Overall description.....	4
2.1	Product perspective.....	4
2.1.1	System interfaces	4
2.1.2	User interfaces	4
2.1.3	Hardware interfaces	4
2.1.4	Software interfaces.....	5
2.1.5	Communications interfaces	5
2.1.6	Memory constraints.....	5
2.1.7	Operations	5
2.2	Product functions	6
2.3	User characteristics.....	7
2.4	Constraints	7
2.5	Assumptions and dependencies	7
2.6	“Nice To Have” Plans	8
3	Specific Requirements	8
3.1	External Interface Requirements	8
3.1.1	User interfaces	8
3.1.2	Hardware interfaces	8
3.1.3	Software interfaces.....	9
3.1.4	Communications interfaces	9
3.2	Functional Requirements.....	9
3.2.1	System Logic Flow	11
3.3	Logical database requirements	12
3.4	Design constraints	12
3.4.1	Standards compliance	12

3.5	Non-Functional Requirements	12
3.5.1	Performance requirements	12
3.5.2	Reliability	13
3.5.3	Availability	13
3.5.4	Security	13
3.5.5	Maintainability	13
3.5.6	Portability	13
4	References	13

1 Introduction

1.1 Purpose

The purpose of the **P2P Dynamic Port Switching System** is to increase communication security by preventing attackers from reliably identifying or targeting the active communication port of a peer.

The system achieves this by:

- Dynamically and periodically changing the TCP or UDP communication port between two peers
- Reducing the effectiveness of port-scanning, reconnaissance, and brute-force attacks
- Providing a decentralized Moving Target Defense (MTD) mechanism without requiring a central controller
- Ensuring that even if an attacker discovers the communication port at a given moment, the port will no longer be valid in the next time interval
- Allowing two peers to securely synchronize port changes using a deterministic randomness algorithm and deterministic time-based algorithm

Thus, the purpose of the project is to provide **unpredictability, and self-adaptive peer-to-peer communication mechanism** that significantly complicates the attacker's ability to locate, analyze, or disrupt the communication channel.

1.2 Scope

The P2P Dynamic Port Switching system is a decentralized communication mechanism where two peers, without any centralized controller, synchronize port changes using a deterministic randomness algorithm and to that specific time-interval. The system alternates peer roles (listener/dialer), computes synchronized ports, authenticates peers, and establishes untraceable communication channels resistant to scanning and reconnaissance attacks.

The software **will**:

- Dynamically computing communication ports based on randomness algorithm (known by peers) + time-interval
- Synchronize two peers in a peer-to-peer setting
- Assign roles (listener / dialer) deterministically
- Log communication events for analysis
- Provide configuration for time interval, window, port range, secret key
- The system is limited with **two peers**.
- **User Interface:** The system is fundamentally designed to operate via **Command-Line Interface (CLI)** to ensure low resource usage and scriptability. A Graphical User Interface (GUI) is designated as a **"nice-to-have" feature** and may be developed to provide visual monitoring if time resources permit.
- **Supported Platforms:** The application is designed to operate on **Unix-based systems (Linux, macOS)** compatible with the TCP/IP protocol suite.

The software **will not**:

- Provide full production-grade distributed networking infrastructure
- Multi-peer/multi-hop topologies, mesh routing, and group communication are not included.
- Implement full encryption channels (TLS) — can be added later
- Guarantee real-world WAN traversal in all NAT types
- Replace existing VPN, firewall, or intrusion detection systems

The **primary goals** are to:

- Demonstrate feasibility of decentralized Moving Target Defense
- Improve port-scanning resistance
- Provide a P2P communication .
- Serve as an academic research prototype
-

1.3 Threat Model and Defense Objectives

This section defines the specific cybersecurity threats the P2P Dynamic Port Switching System is designed to mitigate, as well as the limitations of its defense mechanisms.

1.3.1 In-Scope Threats (Mitigated Attacks)

The primary goal of the system is to implement a Moving Target Defense (MTD) strategy to neutralize the following attacks:

- **Reconnaissance and Port Scanning Attacks:**
 - **Description:** Attackers use tools like Nmap, Masscan, or Zmap to scan a target IP for open ports to identify running services.
 - **Mitigation:** By changing the listening port at short intervals, the system renders scan results obsolete almost immediately. A port discovered as "open" at time t will be "closed/filtered" at time $t+1$.
- **Service Enumeration and Fingerprinting:**
 - **Description:** Attackers attempt to identify the specific application protocol and version running on a port (Banner Grabbing) to select appropriate exploits.
 - **Mitigation:** The "listener" acts as a moving target, preventing the attacker from maintaining a stable connection long enough to perform deep packet inspection or banner grabbing without proper authentication.
- **Targeted Application-Layer DoS:**
 - **Description:** Denial of Service attacks that target a specific static port (e.g., flooding port 80 or 443) to exhaust application resources.
 - **Mitigation:** Since the application does not listen on a fixed port, attack traffic directed at a previously known port is dropped by the OS or firewall (blackholed), protecting the application resources.
- **Replay Attacks:**
 - **Description:** Attackers capture valid data packets or authentication tokens transmitted between peers and attempt to retransmit them later to gain unauthorized access or duplicate actions.
 - **Mitigation:** Since the valid communication port changes at every time interval, a captured packet destined for Port X at current time-interval will be rejected at the next time-interval because the listener has already moved to Port Y. This inherently invalidates old packets without needing complex timestamp checks.

- **Automated Malware & Worm Propagation:**
 - **Description:** Automated malware (worms, botnets) typically scans specific, well-known ports (e.g., SSH on 22, SMB on 445) across a subnet to exploit known vulnerabilities and propagate.
 - **Mitigation:** By operating on a dynamic, random high-number port range (e.g., 20000-60000) rather than fixed standard ports, the system becomes invisible to mass-scanning malware scripts that target default service configurations.

1.3.2 Out-of-Scope Threats (Not Mitigated)

The following threats are acknowledged but are **not** addressed by this prototype:

- **Volumetric DDoS Attacks:** The system cannot prevent Distributed Denial of Service attacks that saturate the network bandwidth (Layer 3/4 flooding) of the target IP address.
- **IP Spoofing:** The system does not validate the source IP address at the network layer, relying instead on application-layer cryptographic authentication.

1.4 Definitions, acronyms, and abbreviations

This subsection should provide the definitions of all terms, acronyms, and abbreviations required to properly interpret the SRS. This information may be provided by reference to one or more appendixes in the SRS or by reference to other documents.

Term	Description
Port-hopping / Dynamic Port Switching	Periodic change of communication ports to evade reconnaissance and scanning attacks.
MTD(Moving Target Defense)	MTD is a cybersecurity approach that increases the security of a system by continuously changing its structure or behavior, making it more difficult for an attacker to predict the target.
P2P	Peer-to-Peer communication architecture without central server
Listener	Peer role responsible for binding to the computed port
Dialer	Peer role responsible for initiating the connection
NAT(Network Address Translation)	This may block inbound connections
Synchronization Secret Key	A static initial value shared between peers used to initialize the port generation algorithm.

1.5 Overview

This SRS's structure is below:

- **Section 2:** System overview, constraints, environment, and product functions
- **Section 3:** Detailed software requirements including interfaces, functional requirements, and performance constraints
- **Section 4:** Reference list

1.6 Version History

Table of changes:

Version No	Description of change	Date
1.0	Initial Release(Literature Review)	06.11.2025
1.1	Second Release(The SRS)	30.11.2025
1.2	Third Release(updated via feedback of the advisor)	26.12.2025

2 Overall description

2.1 Product perspective

The P2P Dynamic Port Switching system is a **standalone research prototype**. It is not a module of a larger distributed framework but may be incorporated into future security systems or MTD engines. It interacts directly with:

- Local OS networking stack
- Network interfaces (LAN/Wi-Fi)

The system runs on user machines (developer laptops or VMs) and operates in an isolated lab environment.

2.1.1 System interfaces

- Operating System: Linux/MacOS
- Networking Stack: TCP/IP sockets
- The system strictly operates within the **TCP/IP protocol suite**. It interfaces with the Transport Layer to manage TCP and UDP sockets directly.

2.1.2 User interfaces

- Command-line interface (**CLI**) is first priority but a **GUI** is nice to have if it is needed in future plans.
- Console logging for debugging and experiment analysis

2.1.3 Hardware interfaces

- Standard **NIC (Network Interface Card)** supporting TCP/IP networking
- Ethernet adapters, Wi-Fi adapters, or virtual NICs (VirtualBox/VMware)
- No specialized hardware required

2.1.4 Software interfaces

The system will interact with the following software components, but programming languages are not limited to just C or Python. The best and most effective of them will be chosen in the future plans:

- C language core implementation
- Optional Python
- Automation tools: Git, VSCode, terminal shells

2.1.5 Communications interfaces

- TCP/IP communication between peers
- Optional UDP according to the packet loss and network congestion
- Local LAN or VM network environment
- No WAN requirement

2.1.6 Memory constraints

- Low memory consumption is expected as the system does not maintain heavy databases.
- No database usage

2.1.7 Operations

Users can:

- Start the peer instance as listener/dialer based on computed role
- Send simple text messages once connection is established
- Changing modes: Automatic Mode or Manual Mode

2.2 Product functions

The primary system functions:

Dynamic Port Generation Strategy

- The system is designed to generate dynamic port numbers synchronously across peers based on the Current Time-Interval and Synchronization Secret Key.
- To achieve this synchronization, the system will employ a deterministic randomness algorithm. The final implementation will be selected from the following candidate strategies, depending on performance and resource constraints:
 - Standard Seeded PRNG (Linear Congruential Generator): Utilizing standard C library functions (e.g., rand()) seeded with a combination of a pre-shared key and the time interval. This is the primary candidate for low-resource environments.
 - Hash-Based Generation (SHA/MD5): Generating ports by hashing the time interval with a secret salt. This offers higher entropy but requires more CPU cycles.
 - TOTP-Like Logic (Time-Based One-Time Password): Implementing a simplified version of the TOTP algorithm adapted for port integers rather than authentication codes.
- The specific algorithm will be determined during the detailed design phase to balance security with latency requirements

Role Determination

- The system determines, for each interval, which peer will act as:
 - **Listener** (binding and waiting for incoming connections), and
 - **Dialer** (actively initiating the connection).
- This decision is made using a deterministic function that takes the shared secret key and each peer's identifier as input. As a result, both peers independently arrive at consistent roles without needing a central coordinator.

Connection Handling

- The system manages the establishment of connections between the two peers as follows:
- The **listener** binds to the computed port for the current interval and waits for incoming connections.
- The **dialer** attempts to connect to the port values computed for the current time-step and a small drift window (e.g., previous/next interval) to tolerate clock differences.
- If a connection attempt fails, the dialer may retry within the configured limits.
- Once a connection is successfully established, the system can exchange control messages and application data over this channel.

Transport Layer Protocols

- The system should support two modes of operation regarding transport protocols:
- **Manual Mode:** User selects either TCP or UDP for the session.
- **Automatic Mode:** The system dynamically switches between TCP and UDP based on network congestion or packet loss metrics.

Logging & Error Handling

- Connection attempts
- Port calculations
- Failures / successes

2.3 User characteristics

The system is intended for:

- Developers familiar with socket programming
- Cybersecurity researchers studying MTD
- Applications integrating the library at code level

Technical background:

- Basic networking
- Knowledge of security concepts (seeds, time-based tokens)

2.4 Constraints

The system is subject to the following constraints:

1. **Time Synchronization Constraint**
 - Both peers must maintain reasonably synchronized clocks.
 - Time drift is tolerated only within ± 1 interval; larger drift breaks synchronization.
 - NTP must be active and functional.
2. **Network Environment Constraint**
 - It is assumed that NAT traversal issues are resolved by the network environment configuration.
3. **Port Range Constraint**
 - Port range must be sufficiently large to ensure unpredictability.
 - Uncommon port range will be used.
4. **Secret-Key Security Constraint**
 - Shared secret key must be protected from exposure.
 - If secret key is compromised, port pattern becomes predictable.
5. **Performance Constraint**
 - Time Interval must not be too short; extremely frequent port changes may destabilize connections.
6. **Programming Language Constraint**
 - Programming language is not limited, but the focus will be between C and Python.
7. **Testing Constraint**
 - All scanning or experimental traffic must be contained within an isolated test network.

2.5 Assumptions and dependencies

- It is assumed that the Synchronization Secret Key is securely exchanged between peers prior to the session (out-of-band)
- Network connectivity exists (TCP/IP)
- Users have permission to run custom scripts on their OS

2.6 “Nice To Have” Plans

Future versions may include:

- GUI interface
- WAN support / STUN server

3 Specific Requirements

This section defines all functional and non-functional requirements of the **P2P Dynamic Port Switching** system in sufficient detail to guide design, implementation, and testing.

3.1 External Interface Requirements

This subsection describes all interactions between the system and external entities, including users, hardware, software components, and communication channels.

3.1.1 User interfaces

The system provides a **command-line interface (CLI)** for configuring and launching a peer instance.

The CLI supports the following **parameters**:

- **--secretKey:**
The shared **initial value or secret key** that both peers must use to initialize the port generation algorithm.
- **--peer-id:**
A unique identifier assigned to each peer; determines the peer’s listener/dialer role per interval.
- **--interval:**
Time duration (in seconds) after which the port changes. Used in time-step calculations.
- **--port-range:**
Specifies the range of allowable ports for port-hopping (e.g., 20000–30000).

The CLI provides:

- Printed logs showing computed ports, roles, and connection attempts
- Error messages for failed connections or authentication failures

3.1.2 Hardware interfaces

- Standard NIC
- Connectivity via Ethernet / Wi-Fi / VM virtual adapters

3.1.3 Software interfaces

- C or/and Python (will be determined later according to conditions)
- Optional NAT tools or firewall rules
- GitHub repo for updates

3.1.4 Communications interfaces

The system communicates using:

- **C-based networking modules**
 - For performance-critical socket operations, lower-level networking logic, or integration with OS-level APIs.
- **TCP/UDP** communications between peers.
- LAN / VM networking environment

3.2 Functional Requirements

FR1 — Port Computation

- The system should compute the active communication port using a deterministic algorithm based on the current time interval.
- The system should support a modular design allowing the underlying randomization logic (e.g., PRNG vs. Hashing) to be selected or replaced without altering the main application flow.
- The algorithm must ensure that two peers with the same time-interval and shared configuration generate the exact same port number.

FR2 — Role Assignment

- The system should determine the role of each peer (listener/dialer).

FR3 — Listener Behavior

- The system should bind to the computed port and accept incoming connections.

FR4 — Dialer Behavior

- The system should try connecting to ports computed for time-step.
- The system should retry with bounded timeout.

FR5 — Synchronization Recovery

- If connection is lost due to time drift, the system should attempt to calculate ports for previous and next time intervals to re-establish connection

FR6 — Logging

- The system should log port values, roles, connection attempts, failures, and authentication results.

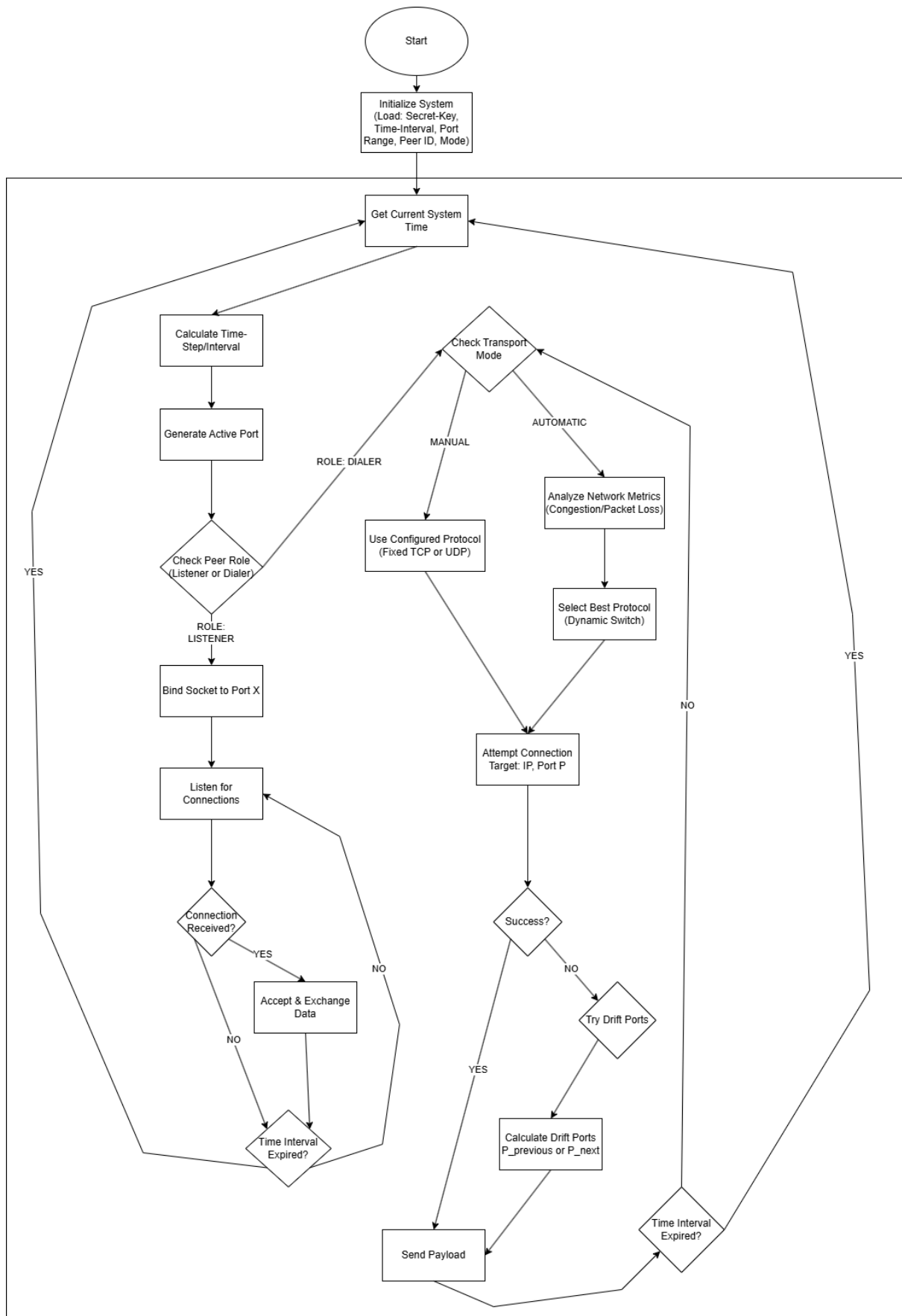
FR7 — Error Handling

- The system should handle network failures gracefully and retry.
- The system should timeout connections that exceed configured limits.

FR8 — Automatic Transport

- The system should allow switching between TCP and UDP protocols either manually or automatically based on defined thresholds.

3.2.1 System Logic Flow



3.3 Logical database requirements

- No persistent database required.
- Optional: store configuration in JSON/YAML
- Optional: store logs in rotating files

3.4 Design constraints

- The core implementation of the P2P Dynamic Port Switching, including all port-hopping, role determination, and message sending/receiving logic, will be written in C, using low-level (raw) socket APIs.
- Python may optionally be used only for supporting tools (e.g., experiment scripts, log analysis, or configuration helpers), but not for the main protocol engine.
- The system should rely on standard OS socket interfaces (POSIX sockets) to remain portable across major desktop operating systems.
- The implementation must avoid platform-specific features that would prevent compilation on standard C compilers.

3.4.1 Standards compliance

- Conform to standard socket APIs provided by the operating system.
- The implementation should adhere to established programming standards for memory management and random number generation.
- Regarding the random number generation mechanism, the most suitable standard will be selected.

3.5 Non-Functional Requirements

This subsection defines key quality attributes of the P2P Dynamic Port Switching system, including reliability, availability, security, maintainability, and portability.

These attributes describe expectations for the overall behavior of the system, beyond specific functions.

They establish measurable non-functional requirements to ensure the system operates robustly, can be maintained efficiently, remains secure against attacks, and can be deployed across different environments.

3.5.1 Performance requirements

- Port computation should take less than 5 ms.
- Authentication token generation should take less than 5 ms.
- Under LAN conditions, connection establishment success rate should be bigger than 90%.

- Drift tolerance should allow ± 1 interval difference.

3.5.2 Reliability

- System should recover from failed connection attempts
- System should handle lost packets
- System should verify drift conditions

3.5.3 Availability

- Listener port must be active during each interval
- System will reset automatically on failures. (WORST CASE)

3.5.4 Security

- The system should validate peers based on the possession of the correct **randomization secret-key**.
- Failed connection attempts due to invalid secret keys should be dropped silently.

3.5.5 Maintainability

- The system should be modular, with clear separation between logic layers.
- **C modules** should be isolated to performance-critical components, with documented APIs to simplify future maintenance.

3.5.6 Portability

- The system should run on platforms that support standard **C** compiler (Linux, macOS).
- All networking functionality should rely on standard OS socket APIs to ensure portability.
- The codebase should avoid OS-specific features unless absolutely necessary.

4 References

- <https://link.springer.com/article/10.1631/FITEE.1601321#:~:text=,383>
- <https://www.mdpi.com/2079-9292/11/18/2886#:~:text=A%20Survey%20on%20Moving%20Target,for%20Networks%3A%20A%20Practical%20View>
- <https://www.mdpi.com/2079-9292/11/18/2886#:~:text=93,CrossRef>
- https://www.usenix.org/legacy/event/usenix05/tech/general/full_papers/ford/ford.pdf#:~:text=USENIX%20Association%202005%20USENIX%20Annual,com%20Dan%20Kegel
- https://beej.us/guide/bgnet/pdf/bgnet_a4_c_1.pdf#:~:text=%5BPDF%5D%20bgnet_a4_c_1.pdf%20,and%20translations%2C%20below%2C%20this
- <https://www.geeksforgeeks.org/c/socket-programming-cc/>
- <https://www.geeksforgeeks.org/computer-networks/p2p-peer-to-peer-file-sharing/>
- <https://www.geeksforgeeks.org/c/tcp-server-client-implementation-in-c/>

- <https://beej.us/guide/bgnet/html/split/#:~:text=Beej%27s%20Guide%20to%20Network%20Programming>
- <https://www.geeksforgeeks.org/computer-networks/nat-hole-punching-in-computer-network/>